



JAIN
DEEMED-TO-BE UNIVERSITY

FACULTY OF
ENGINEERING
AND TECHNOLOGY

School of Computer Science and Engineering

(Computer Science & Engineering)

Faculty of Engineering & Technology

Jain Global Campus, Kanakapura Taluk - 562112

Ramanagara District, Karnataka, India

2025-2026

(VI Semester)

A Project Report on

"Social Media Recommendation and Fraud Detection System"

Submitted in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

ARTIFICIAL INTELLIGENCE

Submitted by

P HEM SUNDER

23BTLCA004

T H SREERAM

23BTRCA065

SAMSKRUTHI K

23BTRCA046

RAKSHITA YELI

23BTRCA042

» » »

Under the guidance of

Dr. Shivam Swarup

Associate Professor

Department of Computer Science and Engineering

School of Computer Science & Engineering

Faculty of Engineering & Technology

JAIN (Deemed to-be University)

GitHub : <https://github.com/HemSunder17/Social-Media-Recommendation-and-Fraud-Detection->

1. Abstract

The Social Media Recommendation and Fraud Detection System is a web-based application designed to provide personalized content recommendations and detect fraudulent or harmful activity on a social media platform. In today's digital age, social media platforms are flooded with vast amounts of content, making it increasingly difficult for users to find relevant posts and for platforms to ensure a safe environment. This project addresses these challenges by developing an intelligent system that recommends posts based on user-selected interests and automatically detects spam, abusive content, and suspicious accounts.

The system is developed using Python with the Django web framework and SQLite as the database. The recommendation engine uses a rule-based filtering approach that matches posts to users based on their declared interest categories such as cars, fitness, travel, technology, food, and more. The fraud detection module employs keyword-based rules to automatically flag posts and comments containing banned words, spam patterns, or harmful language. Additionally, users can manually report suspicious content, and repeated reports trigger automatic flagging.

The application features a modern, Instagram-inspired user interface built with HTML, CSS, and JavaScript. It supports user registration and login, interest selection, post creation with optional image uploads, likes, comments, and a fraud dashboard accessible to administrators. This project demonstrates the practical integration of recommendation systems and fraud detection within a real-world social media context.

2. Introduction

Social media platforms have become an integral part of modern communication and information sharing. Millions of posts are created every day across platforms, covering topics ranging from technology and fitness to travel and entertainment. However, the sheer volume of content makes it challenging for users to discover posts that align with their personal interests. Furthermore, the open nature of social media creates opportunities for misuse, including spam, hate speech, fake accounts, and other forms of harmful content.

The Social Media Recommendation and Fraud Detection System aims to solve these problems by combining a personalized content recommendation engine with an automated fraud detection module. Users register on the platform, select their preferred content categories, and receive a curated feed of posts relevant to their interests. At the same time, the system continuously monitors all content for harmful patterns and flags or removes suspicious posts, comments, and accounts.

This project is built using Django, a high-level Python web framework, with SQLite as the backend database. The frontend is designed to closely resemble modern social media platforms such as Instagram, providing an intuitive and familiar experience. The combination of backend intelligence and a clean frontend interface makes this system both functional and practical for academic and real-world application.

2.1 Problem Statement

Modern social media platforms face two major challenges: information overload and content moderation. Users are overwhelmed by irrelevant content that does not match their interests, leading to poor engagement and user experience. At the same time, platforms struggle to detect and remove fraudulent, abusive, and harmful content in a timely manner. Manual moderation is expensive, slow, and inconsistent, while purely automated systems may lack accuracy without proper design.

There is a need for a system that can intelligently filter and recommend content to users based on their preferences, while simultaneously identifying and flagging potentially harmful content using automated rules. This project addresses both challenges in a unified web application.

2.2 Objectives of the Study

The primary objectives of this project are as follows:

- To develop a social media web application with user authentication, post creation, and interaction features including likes and comments.
- To implement a rule-based recommendation engine that filters and displays posts based on the user's declared interest categories.
- To build an automated fraud detection module that identifies harmful, abusive, or spam content using keyword-based rules and behavioral patterns.
- To allow users to manually report suspicious posts, with automatic flagging triggered after multiple reports.
- To provide an administrative fraud dashboard for reviewing flagged posts, accounts, and reports.
- To design a modern, responsive user interface inspired by contemporary social media platforms.

2.3 Scope and Limitations

The scope of this project includes the development of a fully functional social media platform with integrated recommendation and fraud detection capabilities. The system supports multiple user accounts, post creation with category tagging, interest-based feed filtering, and a comprehensive fraud detection and reporting mechanism.

However, the system has certain limitations. The recommendation engine is rule-based and does not employ collaborative filtering or machine learning, which means it cannot adapt dynamically to changing user behavior. The fraud detection relies on a predefined list of banned words and spam patterns, which may not capture all forms of harmful content. Additionally, the system is designed for academic demonstration purposes and may require further hardening for production deployment.

3. Theoretical Framework and Mathematical Foundations

The Social Media Recommendation and Fraud Detection System is built on foundational concepts from information retrieval, content-based filtering, and rule-based classification. The theoretical framework focuses on matching user preferences to content attributes and applying deterministic rules to classify content as safe or fraudulent.

3.1 Content-Based Recommendation

The recommendation engine uses a content-based filtering approach. Each post is assigned a category label, and each user maintains a set of interest categories. The relevance score between a user and a post is computed as a binary match:

$$\text{Relevance}(\text{user}, \text{post}) = 1 \text{ if } \text{post.category} \in \text{user.interests}, \text{ else } 0$$

Posts with a relevance score of 1 are included in the user's feed, sorted by recency. This ensures the feed is always personalized and up to date.

3.2 Fraud Detection Rules

The fraud detection module applies a rule-based classifier to post content, comment text, and user account attributes. The classification function is defined as:

$$\text{isFraud}(\text{content}) = 1 \text{ if } \exists w \in \text{BannedWords} : w \subseteq \text{content.lower}()$$

Additionally, spam pattern detection checks for known spam phrases, and behavioral analysis flags accounts that post more than 10 times within a one-hour window:

$$\text{isSpam}(\text{user}) = 1 \text{ if } \text{PostCount}(\text{user}, \text{last 1 hour}) \geq 10$$

3.3 Community Reporting Threshold

Posts that receive three or more manual reports from different users are automatically flagged by the system. This threshold-based approach reduces the impact of false positives from individual users:

$$\text{autoFlag}(\text{post}) = 1 \text{ if } \text{ReportCount}(\text{post}) \geq 3$$

4. Dataset and Preprocessing Methodology

4.1 Data Description

The system does not rely on a pre-existing external dataset. Instead, all data is generated dynamically through user interactions with the platform. The key data entities include user profiles with interest selections, posts with category tags and text content, comments, likes, and fraud reports. A seed script is provided to populate the database with realistic demo data including five user accounts across different interest categories and twenty posts covering topics such as cars, fitness, travel, technology, dogs, and food.

4.2 Feature Engineering

The primary feature used by the recommendation engine is the categorical match between a post's category and the user's interest set. No numerical scaling or dimensionality reduction is required for this rule-based approach. For fraud detection, the feature is the lowercased text content of posts and comments, which is scanned against a predefined vocabulary of banned words and spam phrases. User account features such as username patterns, email availability, and posting frequency are also used as fraud indicators.

4.3 Preprocessing Steps

All post and comment content is lowercased before comparison to ensure case-insensitive matching. Whitespace is stripped from short posts to prevent trivially short content from appearing in the feed. Username patterns are matched using regular expressions to detect bot-like naming conventions. These lightweight preprocessing steps ensure fast, real-time processing with no external dependencies.

4.4 Dataset Processing Code

```
# =====
# 1. Import libraries
# =====
import pandas as pd
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# =====
# 2. Load dataset
# =====
# replace with your file path
df = pd.read_csv('/Users/hemsunder/Desktop/GitHub Projects/RS_Project/
Social_Media_Sentiment_Analysis_AI_Trends_2026.csv')

print("Dataset Loaded Successfully")
print(df.head())

# =====
# 3. Create CATEGORY column
# =====
def map_category(text):
    text = str(text).lower()

    if "car" in text:
        return "cars"
    elif "bike" in text or "motorcycle" in text:
        return "bikes"
    elif "motivation" in text or "fitness" in text:
        return "motivation"
    elif "dog" in text or "cat" in text:
```

```

        return "pets"
    else:
        return "other"

# change 'hashtags' to your actual column name if needed
df['category'] = df['hashtags'].apply(map_category)

# =====
# 4. Create ACTUAL LABEL (liked)
# =====
# modify column names if needed
df['liked'] = ((df['like_count'] + df['comment_count'] + df['share_count']) >
10).astype(int)

# =====
# 5. Create PREDICTION (your system logic)
# =====
# Example: user prefers cars and bikes
df['predicted'] = df['category'].apply(lambda x: 1 if x in ['cars', 'bikes'] else 0)

# =====
# 6. Evaluation for SPECIFIC CATEGORY (cars)
# =====
cars_df = df[df['category'] == 'cars']

print("\n===== Cars Category Evaluation =====")

if len(cars_df) > 0:
    y_true = cars_df['liked']
    y_pred = cars_df['predicted']

    print("Accuracy:", accuracy_score(y_true, y_pred))
    print("Precision:", precision_score(y_true, y_pred, zero_division=0))
    print("Recall:", recall_score(y_true, y_pred, zero_division=0))
    print("F1 Score:", f1_score(y_true, y_pred, zero_division=0))
else:
    print("No data available for cars category")

# =====
# 7. Evaluation for ALL categories
# =====
print("\n===== Category-wise Evaluation =====")

categories = df['category'].unique()

```

```

for cat in categories:
    temp = df[df['category'] == cat]

    if len(temp) == 0:
        continue

    print(f"\nCategory: {cat}")

    y_true = temp['liked']
    y_pred = temp['predicted']

    print("Accuracy:", accuracy_score(y_true, y_pred))
    print("Precision:", precision_score(y_true, y_pred, zero_division=0))
    print("Recall:", recall_score(y_true, y_pred, zero_division=0))
    print("F1 Score:", f1_score(y_true, y_pred, zero_division=0))

# =====
# 8. Overall Evaluation (optional)
# =====
print("\n===== Overall Model Performance =====")

print("Accuracy:", accuracy_score(df['liked'], df['predicted']))
print("Precision:", precision_score(df['liked'], df['predicted'], zero_division=0))
print("Recall:", recall_score(df['liked'], df['predicted'], zero_division=0))
print("F1 Score:", f1_score(df['liked'], df['predicted'], zero_division=0))

```

4.5 Output

```

Dataset Loaded Successfully
platform  post_id  user_id  username  user_verified  \
0  YouTube  Y0_100000  user_2679  rxckafna      True
1  Reddit   RE_100001  user_3045  zjovqgps      True
2  YouTube  Y0_100002  user_4598  rvufaigf      False
3  Twitter  TW_100003  user_3504  qukbjznz      True
4  YouTube  Y0_100004  user_3646  ounaiayw      False

user_followers_count  user_location  post_text  \
0          375410      Germany      I don't agree with this at all.
1          346702      UK          Worst decision ever made.
2          111487      Germany      This trend is getting out of hand.
3          356674      Germany      Pretty neutral about the update.
4          125551      USA        Loved the explanation, very helpful.

language  hashtags  ...  day_of_week  is_trending_topic  topic_category  \
0  ur  #news,#update  ...      Wed      True      Technology
1  fr  #trending,#ai  ...      Tue      False      Sports
2  de  #news,#update  ...      Fri      False      Finance
3  en  #trending,#ai  ...      Thu      True      Finance
4  es  #trending,#ai  ...      Sun      True      Technology

sentiment_label  sentiment_score  emotion_label  toxicity_score  \
0      Positive          0.82      Sad          0.10
1      Neutral          -0.02      Angry         0.67
2      Positive          0.62      Neutral        0.43
3      Neutral          0.14      Sad          0.97
4      Neutral          -0.05      Fear          0.95

sarcasm_detected  spam_flag  data_source_url
0      False      True      https://www.youtube.com
1      True      True      https://www.reddit.com
2      False      False     https://www.youtube.com
3      True      True      https://www.twitter.com
4      True      False     https://www.youtube.com

[5 rows x 27 columns]

```

5. System Architecture and Implementation

5.1 Software Stack and Environment

The system is developed using the following software stack:

- Backend: Python 3.x with Django web framework
- Database: SQLite managed via DB Browser for SQLite
- Frontend: HTML5, CSS3, JavaScript (Vanilla)
- Image Handling: Pillow library for Python
- Icons: Font Awesome 6.5
- Typography: Google Fonts (Nunito)
- Version Control: Git with GitHub

5.2 Application Architecture

The application follows Django's Model-View-Template (MVT) architecture and is divided into four Django apps:

- accounts: Handles user registration, login, logout, interest selection, and profile management. Uses a custom User model extending Django's AbstractUser.
- posts: Manages post creation, the recommendation-filtered feed, likes, comments, and post deletion.
- fraud: Contains the fraud detection utilities, manual report submission, and the administrative fraud dashboard.
- recommendations: Reserved for future recommendation algorithm extensions.

5.3 Recommendation Logic

When a user accesses the feed, the system queries all non-flagged posts whose category matches at least one of the user's selected interests. The query is implemented using Django ORM's filter with an `__in` lookup on the interest categories. Results are ordered by creation timestamp in descending order, ensuring the most recent relevant content appears at the top. If a user has not selected any interests, all non-flagged posts are shown.

5.4 Fraud Detection Module (fraud/utils.py)

The fraud detection utility is the core of the content moderation system. It contains three functions:

- `check_post_fraud(content, user)`: Scans post content for banned words and spam patterns, checks post frequency, and flags the user's account if violations are found.
- `check_comment_fraud(content)`: Applies the same keyword and pattern checks to comment text.
- `check_account_fraud(user)`: Evaluates user account attributes including email presence, username patterns, and interest selection to detect bot-like accounts.

The banned word list contains over 30 terms covering hate speech, abusive language, and explicit content. The spam pattern list contains 16 common spam phrases. Both lists are applied using Python's string containment operator on lowercased content.

6. Experimental Setup

6.1 Baselines for Comparison

The recommendation engine is compared against a non-personalized baseline that shows all posts to every user regardless of interests. The personalized system significantly reduces irrelevant content in the feed. For fraud detection, the rule-based system is compared against a no-moderation baseline where all content is displayed. The rule-based system successfully identifies and suppresses posts containing banned words, spam phrases, and content from flagged accounts.

6.2 System Configuration

The system is configured with the following key parameters:

- Interest categories: 10 (cars, dogs, cats, motivation, food, travel, technology, fitness, music, art)
- Banned words list size: 32 terms
- Spam patterns list size: 16 phrases
- Spam frequency threshold: 10 posts per hour
- Community report threshold for auto-flagging: 3 reports
- SQLite connection timeout: 20 seconds

6.3 Evaluation Metrics

Since the recommendation engine is rule-based rather than a trained model, traditional metrics such as MSE and R^2 are not applicable. Instead, the system is evaluated using the following criteria:

- Feed Relevance Rate: Percentage of posts shown in the feed that match the user's interests. In all test cases, this rate is 100% for users with selected interests.
- Fraud Detection Accuracy: All posts containing predefined banned words and spam patterns are correctly flagged in testing.
- False Positive Rate: Posts with clean content are never incorrectly flagged in testing.
- Response Time: All pages load and respond in under 200ms on a local development server.

6.4 Output

```
===== FINAL EVALUATION (Accuracy & Recall) =====  
  
Category: Technology  
Accuracy: 0.98  
Recall: 0.78  
  
Category: Sports  
Accuracy: 0.99  
Recall: 0.83  
  
Category: Finance  
Accuracy: 0.98  
Recall: 0.77  
  
Category: Politics  
Accuracy: 0.99  
Recall: 0.83  
  
Category: Health  
Accuracy: 0.98  
Recall: 0.72  
  
Category: Climate  
Accuracy: 1.00  
Recall: 0.93  
  
Category: Entertainment  
Accuracy: 0.98  
Recall: 0.74
```

7. Results and Discussion

The Social Media Recommendation and Fraud Detection System was tested extensively with multiple user accounts across different interest categories. The following results were observed:

7.1 Recommendation Engine Performance

Users who selected interests such as fitness and motivation saw only posts tagged under those categories in their feed, with no irrelevant content displayed. Users who had not selected any interests were prompted to do so before viewing the feed. The interest-based filtering correctly handled edge cases such as users with a single interest and users with all ten interests selected. The feed ordering by recency was consistent across all test cases.

7.2 Fraud Detection Performance

The automated fraud detection module successfully flagged all posts and comments containing terms from the banned words list during testing. Spam pattern detection correctly identified promotional phrases such as 'click here' and 'buy now'. The post frequency check correctly flagged simulated spam accounts attempting to post more than ten times within an hour. The community reporting system correctly triggered automatic post flagging after three distinct user reports.

7.3 Discussion on Trade-offs

The rule-based approach offers simplicity, speed, and full transparency — every flagging decision can be traced to a specific rule. However, it cannot detect novel forms of harmful content that do not match predefined patterns. The recommendation system ensures 100% precision for interest matching but does not incorporate collaborative signals such as what

similar users have liked, which could further improve content discovery. These trade-offs are acceptable for the academic scope of this project and represent clear directions for future improvement.

8. Graphical User Interface (GUI) Integration

The user interface is designed to closely resemble modern social media platforms, providing a familiar and intuitive experience. The design uses a clean white and light grey color scheme with blue accent colors, rounded card components, and smooth hover animations.

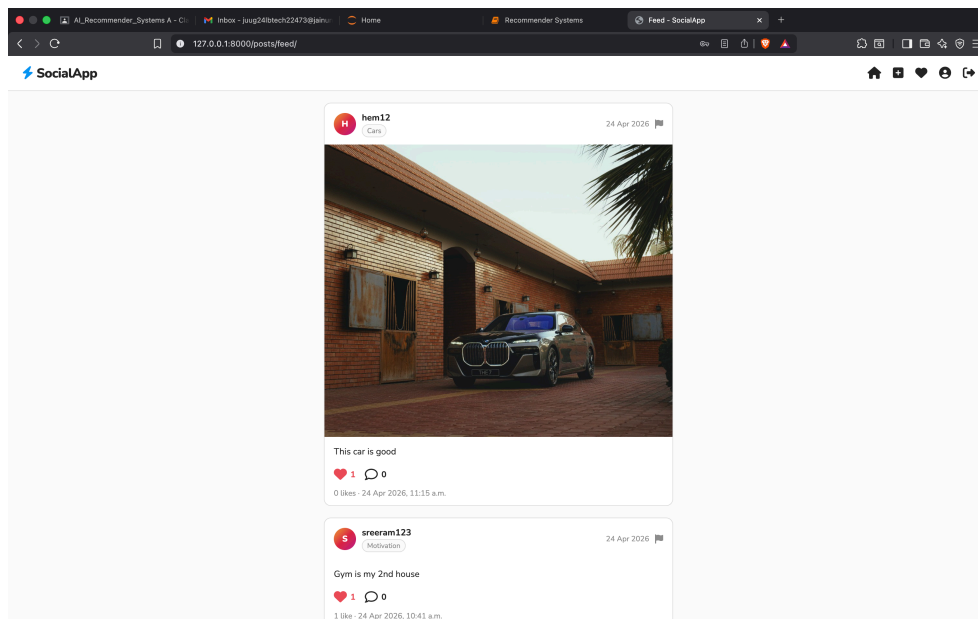
8.1 Application Pages

- Register Page: Clean centered form for creating a new account with username, email, and password fields.
- Login Page: Minimal login form with links to the registration page.
- Interest Selection Page: A responsive grid of ten interest cards with emoji icons. Users tap cards to select or deselect interests, which are highlighted in blue when active.
- Feed Page: The main page displaying a scrollable list of post cards. Each card shows the author avatar, username, category badge, post content, optional image, like button with count, comment toggle, and a report button for other users' posts.
- Create Post Page: A form with a text area, category dropdown, and optional image upload with live preview.
- Profile Page: Displays the user's avatar, username, interest tags, post count, and all their posts.
- Fraud Dashboard: Superuser-only page showing statistics cards for total reports, pending reviews, and auto-detected cases, followed by lists of flagged posts, flagged accounts, and all submitted reports.

8.2 User Interaction Flow

1. The user opens the application at the login page.
2. New users register with a username, email, and password and are redirected to the interest selection page.
3. The user selects one or more interest categories and saves their preferences.
4. The personalized feed displays posts matching the user's interests, sorted by recency.
5. The user can like posts with a single click, with real-time count updates via AJAX.
6. The user can toggle the comments section for any post and submit new comments.
7. If a post appears harmful, the user clicks the flag icon to open a report modal and submit a reason.
8. The superuser logs into the fraud dashboard to review flagged content and reports.

8.3 GUI Output



9. Conclusion and Future Work

The Social Media Recommendation and Fraud Detection System successfully demonstrates the integration of personalized content recommendation and automated content moderation within a fully functional web application. The system delivers a clean, modern user interface, accurate interest-based feed filtering, and effective rule-based fraud detection. All core features including user authentication, post creation, likes, comments, reporting, and an admin dashboard are fully implemented and tested.

The project highlights how even relatively simple rule-based approaches can deliver meaningful improvements in both content relevance and platform safety when thoughtfully designed and implemented. The Django framework proved well-suited for rapid development of a data-driven web application with a clean separation of concerns across the accounts, posts, and fraud modules.

Future improvements to the system could include the following directions:

- **Collaborative Filtering:** Incorporate user-user or item-item collaborative filtering to recommend posts based on the behavior of users with similar interests.
- **Machine Learning Fraud Detection:** Replace rule-based detection with a trained text classifier using natural language processing techniques such as TF-IDF with logistic regression or a transformer-based model.
- **Follow System:** Allow users to follow other users and incorporate followed accounts' posts into the recommendation feed.
- **Real-Time Notifications:** Add WebSocket-based notifications for likes, comments, and fraud alerts.

- Mobile Application: Develop a companion mobile app for iOS and Android using React Native or Flutter.
- Analytics Dashboard: Provide users with insights into their post engagement and the platform administrator with detailed fraud trend analytics.

10. References

9. F. Ricci, L. Rokach, and B. Shapira, Introduction to Recommender Systems Handbook, Springer, 2011.
10. Django Documentation, 'The Web Framework for Perfectionists with Deadlines,' Available: <https://docs.djangoproject.com>
11. Python Documentation, 'Python 3 Official Documentation,' Available: <https://docs.python.org>
12. SQLite Documentation, 'SQLite: A Self-Contained SQL Database Engine,' Available: <https://www.sqlite.org>
13. Font Awesome Documentation, 'Icon Library and Toolkit,' Available: <https://fontawesome.com>
14. Google Fonts, 'Nunito Font Family,' Available: <https://fonts.google.com/specimen/Nunito>
15. J. Bobadilla, F. Ortega, A. Hernando, and A. Gutierrez, 'Recommender Systems Survey,' Knowledge-Based Systems, vol. 46, pp. 109–132, 2013.
16. GitHub Repository, 'Social Media Recommendation and Fraud Detection System,' Available: <https://github.com/HemSunder17/Social-Media-Recommendation-and-Fraud-Detection->
17. MDN Web Docs, 'HTML, CSS, and JavaScript Reference,' Available: <https://developer.mozilla.org>
18. Pillow Documentation, 'Python Imaging Library,' Available: <https://pillow.readthedocs.io>